
Dispatch Overhead in POSIX Shell: An Empirical Measurement of `if/[` versus `case` Across Shell Implementations

Ivan Gaydardzhiev

Independent Research 2026

Abstract

A companion paper argued on structural grounds that `if [ ]` is a category error for value-based dispatch in POSIX shell: it routes a value through a command evaluator, incurring an encoding round-trip that `case` does not require. This paper measures the cost of that round-trip empirically. Three shell configurations were benchmarked on an ARM Cortex-A53 octa-core system running 32-bit Arch Linux: GNU Bash 5.2.37, a self-compiled DASH (armv8l), and the Pacman-distributed DASH binary. Each configuration executed a controlled dispatch loop of 100 000 iterations under both `if/[` and `case`, with arithmetic work held constant. The `case` construct outperformed `if/[` in every run across all three shells, with mean speedup ratios of 2.33×, 2.35×, and 3.09× respectively. Standard deviations were low and ratios consistent, indicating that the overhead is structural and not a measurement artefact. The most significant finding is directional: the faster the shell implementation, the larger the relative overhead of `if/[`. Optimising the shell does not close the gap; it widens it.

1 Introduction

The structural argument for preferring `case` over `if/[` for value-based dispatch in POSIX shell has been made elsewhere [1]. That argument does not require empirical support: the round-trip is mandated by the specification, and its cost is a property of the idiom, not of any particular implementation. Any conformant shell must execute the guard of an `if` statement as a command and inspect its exit status. That requirement cannot be optimised away.

This paper nevertheless measures the cost, for two reasons.

First, a reader who accepts the structural argument on its own terms may still want to know the order of magnitude. The difference between a theoretically unnecessary step and a practically significant one is not always obvious from first principles.

Second, the measurement reveals a non-obvious property of the overhead: it does not diminish as the shell implementation improves. The ratio between `if/[` and `case` is stable

or larger in the faster shell. This is consistent with the structural argument: the round-trip cost is irreducible, so a faster shell reduces both times proportionally, leaving the ratio intact or worsened. But it is not a result one would confidently predict without measuring.

2 Methodology

2.1 Hardware and System

All measurements were taken on a single machine: a MediaTek MT6765 (Helio G85) system-on-chip with an octa-core ARM Cortex-A53 processor, revision 4, running at nominally 26.08 BogoMIPS per core. The system ran 32-bit Arch Linux ARM in armv8l compatibility mode, kernel reporting **ARMv8 Processor rev 4 (v8l)**. Physical memory was 2869 MB. At the time of measurement, approximately 1048 MB was available; the system was under normal interactive load.

The ARM Cortex-A53 is an in-order pipeline with no out-of-order execution. This makes timing more deterministic than a superscalar desktop core and reduces the variance that would require a larger sample to characterise.

2.2 Shell Configurations

Three shell configurations were benchmarked:

1. **Bash 5.2.37**, GNU Bash release (armv7l-unknown-linux-gnueabi), invoked as `bash compare.sh`.
2. **dash-armv8l**, a DASH binary compiled by the author from source for the armv8l target, invoked as `./dash-armv8l compare.sh`.
3. **dash (Pacman)**, the DASH binary distributed by the Arch Linux ARM package manager, invoked as `/usr/bin/dash compare.sh`.

The two DASH configurations represent different compilation paths of the same interpreter. Their differing absolute times reflect compiler and optimisation differences; their differing ratios are the analytically interesting result.

2.3 Benchmark Design

The benchmark script, `compare.sh`, defines two functions: `ftestif` and `ftestcase`. Both iterate a loop of 100 000 steps. In each iteration, both functions compute:

```
r=$((i % 6))
```

exactly once, assigning the result to `r`. The arithmetic work is therefore identical between the two functions. The only variable is how `r` is used for dispatch.

`ftestif` dispatches via `if/[:`

```
if [ "$r" -eq 0 ]; then
    result="even and divisible by three"
elif [ "$r" -eq 2 ] || [ "$r" -eq 4 ]; then
    result="even"
elif [ "$r" -eq 3 ]; then
    result="divisible by three"
else
    result="odd"
fi
```

`ftestcase` dispatches via `case`:

```

case $r in
  0)    result="even and divisible by three" ;;
  2|4)  result="even"                        ;;
  3)    result="divisible by three"          ;;
  *)    result="odd"                        ;;
esac

```

Timing used `date +%s%N` before and after each loop. Each configuration was run four times in sequence without rebooting, producing four paired measurements of `if` time and `case` time per shell. No CPU pinning or process priority adjustment was applied; the measurements reflect conditions representative of normal use.

2.4 A Note on `if/[` Invocation Count

In the worst case, a value that does not match the first or second guard, `ftestif` invokes `[` up to four times per iteration: once for the `if` guard, once for the `[` inside the `||` disjunction of the first `elif`, and once each for the remaining `elif` guards. `ftestcase` invokes no external or built-in command for dispatch at any point. The benchmark does not attempt to normalise per-invocation cost; it measures the total cost of expressing the dispatch problem in each idiom as a programmer would naturally write it.

3 Results

Raw timing data across all four runs per shell are presented in Table 1. Summary statistics (mean, standard deviation, ratio, and mean absolute speedup) are presented in Table 2.

Table 1: Raw timing data (nanoseconds), four runs per shell

Shell	Run	if time (ns)	case time (ns)
dash-armv8l (self-compiled)	1	7 007 660 624	3 134 303 622
	2	7 098 621 941	3 011 229 225
	3	7 097 430 018	3 009 546 762
	4	7 043 410 166	2 969 579 374
bash 5.2.37 (armv7l)	1	6 963 661 004	2 953 720 680
	2	7 047 592 090	3 012 825 302
	3	6 934 439 233	2 974 111 913
	4	7 058 227 783	2 986 966 068
dash (Pacman)	1	2 506 996 943	818 304 389
	2	2 518 489 405	800 539 080
	3	2 497 728 250	808 197 311
	4	2 493 949 788	811 478 312

Table 2: Summary statistics across four runs per shell

Shell	if (s)		case (s)		Ratio	Speedup (s)
	Mean	SD (ms)	Mean	SD (ms)		
dash-armv8l	7.0618	44.33	3.0312	71.40	2.330	4.031
bash 5.2.37	7.0010	61.29	2.9819	24.74	2.348	4.019
dash (Pacman)	2.5043	10.94	0.8096	7.38	3.093	1.695

3.1 Consistency

Standard deviations are low relative to the means in all cases. For the Pacman DASH, the `if` standard deviation is 10.94 ms against a mean of 2.504 s, a coefficient of variation below 0.5%. The `case` standard deviation for the self-compiled DASH is the largest at 71.40 ms against 3.031 s, still under 2.4%. The measurements are stable. The ratios are not noise.

3.2 The Inverse Scaling Result

The most significant finding is not the magnitude of the overhead but its behaviour across implementations.

Bash and dash-armv8l have nearly identical absolute `if` times (7.001 s and 7.062 s respectively), despite being different interpreters. Their `case` times are also close: 2.982 s and 3.031 s. The ratios are 2.348 $\times$ and 2.330 $\times$.

The Pacman DASH is substantially faster in absolute terms: 2.504 s for `if`, 0.810 s for `case`. But its ratio is 3.093 $\times$, higher than either of the slower shells.

This means that a better shell implementation reduced the `case` time more than it reduced the `if` time. The overhead attributable to the `[` round-trip did not shrink proportionally. The irreducible cost of argument vector construction, built-in dispatch, and exit-code propagation was compressed less by the optimisation than the internal dispatch path of `case` was.

The implication is that optimising the shell does not close the semantic gap; it exposes it more clearly.

4 Discussion

4.1 What the Measurement Establishes

These results establish three things.

First, the overhead of `if/[` for value-based dispatch is not theoretical. On the test hardware, under bash, it costs approximately 4 s per 100 000 iterations compared to `case`. At the per-iteration level, that is roughly 40 μ s of avoidable overhead per dispatch, paid entirely to satisfy the interface contract of a command evaluator on a problem that does not involve commands.

Second, the overhead is consistent across implementations. The ratio does not depend on which shell is used or how it was compiled. Every shell tested showed `case` faster by at least 2.3 $\times$. The structural argument predicts this: the cost is in the protocol, not the implementation.

Third, and most importantly, optimising the shell widens the gap. This is not a result that follows immediately from the structural argument alone. It requires measurement to observe. The Pacman DASH, the fastest shell in the test, shows the largest ratio. A programmer who believed that a sufficiently optimised shell would make the distinction irrelevant is empirically wrong.

4.2 Limitations

Four runs per shell is a small sample. The standard deviations are low enough that the ratios are credible, but a larger study with process isolation, CPU pinning, and multiple machines would produce more defensible confidence intervals.

All measurements were taken on a single architecture: 32-bit ARM in-order pipeline. The Cortex-A53's in-order execution makes timing more stable than an out-of-order desktop core, but it may not represent the relative overhead on x86_64 or aarch64 systems. The absolute times would differ; whether the ratios hold is an open empirical question.

The benchmark measures the total cost of dispatch as written, not the per-invocation cost of `[]` in isolation. Isolating the per-invocation cost would require a different experimental design.

These limitations do not affect the primary claim: the overhead is real, it is consistent, and it does not diminish as the implementation improves.

5 Conclusion

The `if/[]` idiom for value-based dispatch in POSIX shell incurs a measurable and consistent overhead relative to `case`. On three shell configurations tested on ARM Cortex-A53 hardware, `case` was faster by factors of $2.33\times$, $2.35\times$, and $3.09\times$. The overhead is structural: it follows from the execution model of `if` and cannot be eliminated by implementation quality. The evidence for this is in the direction of the ratio under optimisation. The gap does not close. It grows.

Ivan Gaydardzhiev, Independent Research, 2026. This work is licensed under the Creative Commons Attribution 4.0 International License (CC BY 4.0).

References

- [1] I. Gaydardzhiev, *Exit Codes Are Not Booleans: On the Misuse of if in POSIX Shell Scripting*, Independent Research, 2026.